

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Development

Java, javac , classpath. Jar , static import

Total Questions : 18

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Developement

Questions Question - 1

You have a java file name Test.java inside javaproject directory.

you have issued below commands from command prompt.

```
cd javaproject
javac Test.java
```

In which of the following directory Test.class will be created ?

- 1.javaproject
- 2.classes folder inside javaproject
- 3.bin folder inside javaproject
- 4.Compilation fails

Explanation :

A is the correct answer.

By default the compiler puts .class files in the same directory as the .java source files present.

Question - 2

You have a java file name Test.java inside src folder of javaproject directory.

You have also classes folder inside javaproject directory.

you have issued below command from command prompt.

```
cd javaproject
```

Which of the below command puts Test.class file inside classes folder ?

- 1.javac -d classes src/Test.java
- 2.javac Test.java
- 3.javac src/Test.java
- 4.javac classes src/Test.java

Explanation :

A is the correct answer.

The -d option lets you tell the compiler in which directory to put the .class file it generates (d for destination)

Question - 3

You have a java file name **Test.java** inside **src** folder of **javaproject** directory.

You have also **classes** folder inside **javaproject** directory.

you have issued below command from command prompt.

```
cd javaproject
javac src/Test.java
```

In which of the following directory **Test.class** will be created ?

- 1.javaproject
- 2.classes folder inside javaproject
- 3.src folder inside javaproject
- 4.Compilation fails

Explanation :

C is the correct answer.

By default the compiler puts .class files in the same directory as the .java source files present.

Question - 4

You have a java file name **Test.java** inside **src/exam** folder of **javaproject** directory.

You have also **classes** folder inside **javaproject** directory.

and **Test.java** source code looks like

```
package exam;
public class Test{

    public static void main (String[] args){
        System.out.println("Hello Test");
    }
}
```

you have issued below command from command prompt.

```
cd javaproject
javac -d classes src/exam/Test.java
```

In which of the following directory **Test.class** will be created ?

- 1.javaproject
- 2.classes folder inside javaproject

3.classes/exam folder inside javaproject

4.Compilation fails

Explanation :

C is the correct answer.

The -d option lets you tell the compiler in which directory to put the .class file it generates (d for destination). In this case package name mentioned "exam" so Test.class file will be created in classes/exam folder inside javaproject.

Question - 5

You have a java file name Test.java inside src/exam folder of javaproject directory.

You have also classes folder inside javaproject directory.

and Test.java source code looks like

```
package exam;
public class Test{

    public static void main (String[] args){
        System.out.println("Hello Test");
    }
}
```

you have issued below command from command prompt.

```
cd javaproject
javac -d classes1 src/exam/Test.java
```

In which of the following directory Test.class will be created ?

1.javaproject

2.classes1 folder inside javaproject

3.classes1/exam folder inside javaproject

4.Compilation fails- system cannot find the path specified

Explanation :

D is the correct answer.

The -d option lets you tell the compiler in which directory to put the .class file it generates (d for destination). In this case classes1 folder does not exist so throw error says "system cannot find the path specified".

Question - 6

You have a class file name Test.class inside javaproject directory.

Test.java source code is :

```
public class Test{

    public static void main (String[] args){
        System.out.println("Hello Test");
    }
}
```

you have issued below commands from command prompt.

```
cd javaproject
java Test.class
```

What is the output ?

- 1.Hello Test
- 2.args Hello Test
- 3.Run successfully but no output
- 4.Run fails - class not found

Explanation :

D is the correct answer.

java command assumes you are executing a .class file, so you don't specify the .class extension on the command line.

Question - 7

You have two class files name `Test.class` and `Test1.class` inside `javaproject` directory.

`Test.java` source code is :

```
public class Test{

    public static void main (String[] args){
        System.out.println("Hello Test");
    }
}
```

`Test1.java` source code is :

```
public class Test1{

    public static void main (String[] args){
        System.out.println("Hello Test1");
    }
}
```

you have issued below commands from command prompt.

```
cd javaproject
java Test Test1
```

What is the output ?

- 1.Hello Test
- 2.Hello Test Hello Test1
- 3.Hello Test1
- 4.Run fails - class not found

Explanation :

A is the correct answer.

You must specify exactly one class file to execute. If more than one then first one will be executed.

Question - 8

You have a class file name `Test.class` inside `javaproject` directory.

`Test.java` source code is :

```
import java.util.Properties;
class Test {
    public static void main (String[] args){
        Properties p = System.getProperties();
        System.out.println(p.getProperty("key1"));
    }
}
```

you have issued below commands from command prompt.

```
cd javaproject
java -Dkey1=value1 Test
```

What is the output ?

- 1.value1
- 2.null
- 3.Run successfully but no output
- 4.Run fails - class not found

Explanation :

A is the correct answer.

`java -D` option from the command line add `key=value` to the system's properties. `java -Dkey1=value1 Test.class`, added `key1` as a key and `value1` as a value to the system's properties, So `p.getProperty("key1")` return `value1`.

Question - 9

You have a class file name `Test.class` inside `javaproject` directory.

Test.java source code is :

```
import java.util.Properties;
class Test {
    public static void main (String[] args){
        Properties p = System.getProperties();
        System.out.println(p.getProperty("key1"));
    }
}
```

you have issued below commands from command prompt.

```
cd javaproject
java -Dkey1="value1 new" Test
```

What is the output ?

- 1.value1 new
- 2.null
- 3.Run successfully but no output
- 4.Run fails - class not found

Explanation :

A is the correct answer.

java -D option from the command line add key=value to the system's properties.
double quotation also allowed in key and value.

Question - 10

You have a class file name Test.class inside javaproject directory.

Test.java source code is :

```
import java.util.Properties;
class Test {
    public static void main (String[] args){
        Properties p = System.getProperties();
        System.out.println(p.getProperty("key1"));
    }
}
```

you have issued below commands from command prompt.

```
cd javaproject
java -D key1=value1 Test
```

What is the output ?

- 1.value1
- 2.null
- 3.Run successfully but no output
- 4.Run fails - java.lang.NoClassDefFoundError: key1=value1

Explanation :

D is the correct answer.

-D option , name=value pair must follow immediately, no spaces allowed. In this case there is space between -D and key1=value1 So java.lang.NoClassDefFoundError:
key1=value1.

Question - 11

You have a class file name **Test.class** inside javaproject directory.

Test.java source code is :

```
class Test {  
    public static void main (String... args){  
        System.out.println(args[1]);  
    }  
}
```

you have issued below commands from command prompt.

```
cd javaproject  
java Test abc xyz pqr
```

What is the output ?

- 1.abc
- 2.xyz
- 3.pqr
- 4.Run fails

Explanation :

B is the correct answer.

args array start with 0 index So args[1] is xyz.

Question - 12

Which of the below statement is true about classpath ?

- 1.A classpath can be declared as a command-line using -classpath.
- 2.A classpath can be declared as an operating system environment variable.
- 3.Classpaths declared as command-line override the classpath declared in environment variable.
- 4.All of the above are true

Explanation :

D is the correct answer.

All statements are true

Question - 13

java and javac command search for other classes that will be necessary to complete the operation.
Which of the below statement is true about search algorithm?

- 1.First search on same directory , if not found then search in classpath
- 2.First search in classpath if not found then search in the same directory
- 3.Not any specific algorithm, can be anywhere
- 4.All of the above are true

Explanation :

A is the correct answer.

First search on same directory if not found then search in classpath. If found then stop searching.

Question - 14

Which of the following statement is true about jar command?

- 1.The jar command creates the META-INF directory implicitly.
- 2.The jar command creates the MANIFEST.MF file implicitly.
- 3.The jar command would not place any of your files in META-INF directory.
- 4.All of the above are true

Explanation :

A is the correct answer.

All statements are true.

Question - 15

You have a java file name Test.java .

Test.java needs access to a class contained in app.jar in "exam" directory.

Which of the following command set classpath to compile clean?

- 1.javac -classpath exam/app.jar Test.java
- 2.javac -classpath app.jar Test.java
- 3.javac -classpath exam Test.java
- 4.None of the above

Explanation :

A is the correct answer.

`javac -classpath exam/app.jar Test.java` is the correct command to set `exam/app.jar` in classpath.

Question - 16

You have a java file name `Test.java` .

`Test.java` needs access to a class contained in "exam" directory.

Which of the following command set classpath to compile clean?

1. `javac -classpath exam/app.jar Test.java`
2. `javac -classpath app.jar Test.java`
3. `javac -classpath exam Test.java`
4. None of the above

Explanation :

C is the correct answer.

`javac -classpath exam Test.java` is the correct command to set `exam` directory in classpath.

Question - 17

What is the output for the below code ?

```
import static java.lang.Math.*;

public class Test {

    public static void main(String... args) {
        double r = cos(PI * 3);
        System.out.println(r);
    }
}
```

1. Compile error at `import static java.lang.Math.*;`
2. Compile error at `double r = cos(PI * 3);`
3. Compile lean
4. None of the above

Explanation :

C is the correct answer.

J2SE 5.0 onwards it allows static import like `import static java.lang.Math.*;` and can use only `PI` no need `Math.PI`

Question - 18

What is the output for the below code ?

```
package bean;

public class Abc {
    public static int index_val = 10;
}

package com;
import static bean.Abc.index_val;

public class Test1 {

    public static void main(String... args) {
        System.out.println(index_val);
    }
}
```

- 1.10
- 2.compile error, index_val not defined
- 3.Compile error at import static bean.Abc.index_val;
- 4.None of the above

Explanation :

A is the correct answer.

The static import construct allows unqualified access to static members without inheriting from the type containing the static members. J2SE 5.0 onwards it allows static import like `import static bean.Abc.index_val;` and can be use directly `System.out.println(index_val);`